# 1. Przygotowanie danych

Proces przygotowania danych do celów badawczych obejmował integrację rzeczywistych chmur punktów LiDAR ze zbiorami syntetycznymi oraz ich ustrukturyzowany podział na podzbiory procesowe.

## 1.1. Charakterystyka wykorzystanych zbiorów danych

W projekcie wykorzystano dwa główne źródła danych:

1. **Zbiór ForInstance v1**:
    - Repozytorium danych: Zenodo (8287792)
    - Artykuł: arXiv:2309.01279
2. **Zbiór syntetyczny**:
    - **Środowisko generacji**: Środowisko grafiki 3D Blender sterowane autorskim skryptem Python.
    - **Geometria obiektów**: Do budowy scen wykorzystano modele 3D drzew wygenerowane w oprogramowaniu TreeIt.

## 1.2. Data Splitting

Zbiór danych ForInstance v1 został poddany podziałowi na trzy rozłączne części: treningową, walidacyjną oraz testową. Zastosowany podział został zrobiony bez większej analizy zawartości poszczególnych scen, bazował bardziej na takim wyborze, żeby każda paczka danych miała swoją reprezentację w treningu i walidacji/teście.

### 1.2.1. Podzbiór Treningowy (TRAIN) – 19 scen

- CULS_plot_1_annotated.las
- NIBIO_plot_1_annotated.las
- NIBIO_plot_2_annotated.las
- NIBIO_plot_4_annotated.las
- NIBIO_plot_5_annotated.las
- NIBIO_plot_7_annotated.las
- NIBIO_plot_8_annotated.las
- NIBIO_plot_10_annotated.las
- NIBIO_plot_11_annotated.las
- NIBIO_plot_13_annotated.las
- NIBIO_plot_16_annotated.las
- NIBIO_plot_18_annotated.las
- NIBIO_plot_19_annotated.las
- NIBIO_plot_22_annotated.las
- RMIT_train.las
- SCION_plot_31_annotated.las
- SCION_plot_39_annotated.las
- SCION_plot_87_annotated.las
- TUWIEN_train.las

### 1.2.2. Podzbiór Walidacyjny (VAL) – 4 sceny

- NIBIO_plot_3_annotated.las
- NIBIO_plot_9_annotated.las
- NIBIO_plot_17_annotated.las
- SCION_plot_35_annotated.las

#### 1.2.3. Podzbiór Testowy (TEST) – 7 scen

- NIBIO_plot_12_annotated.las
- NIBIO_plot_21_annotated.las
- NIBIO_plot_23_annotated.las
- NIBIO_plot_6_annotated.las
- RMIT_test.las
- SCION_plot_61_annotated.las
- TUWIEN_test.las

# 2. Preprocessing

Proces przygotowania danych do treningu został zrealizowany za pomocą czterech dedykowanych notatników Jupyter, umieszczonych w katalogu `preprocessing`. Celem działań jest redukcja/optymalizację objętości danych oraz redukcja i zmiananumeracji klas.

## 2.1. Notatnik 01: Indeksowanie danych (`01_data_indexing`)

Pierwszym etapem było nadanie unikalnego identyfikatora (ID) każdemu punktowi w obrębie paczki danych, celem zapewnienia korelacji między punktami wejściowymi a wynikami segmentacji. Docelowo pomysł na służyć przypisaniu wyników predykcji do oryginalnych chmur punktów na etapie postprocessingu.

## 2.2. Notatnik 02: Ciecie datasetu (`02_data_tiling.ipynb`)

Etap ten obejmował podział wielkoskalowych scen na mniejsze patche/kafelki do przetwarzania przez sieci neuronowe.

### Parametry geometryczne cięcia:

- **Rozmiar kafelka (PATCH_SIZE)**: 20.0 m x 20.0 m.
- **Krok przesunięcia (STRIDE)**: 10.0 m (50% zakładki między sąsiednimi kafelkami).
- **Metoda**: Proste cięcie w układzie siatki kwadratów (Bounding Box mask), bez zaawansowanej filtracji obiektów brzegowych.

### Selekcja i filtracja punktów:

- **Wykluczenie klasy 3 (Out-points)**: Usunięcie punktów sklasyfikowanych jako "Out Points" (https://arxiv.org/pdf/2309.01279), które często reprezentowały fragmenty drzew na krawędziach scen lub artefakty.
- **Limity liczności (MIN/MAX_TOTAL_POINTS)**:
  - Kafelki zawierające mniej niż **100 punktów** były odrzucane jako nieistotne statystycznie.
  - W przypadku kafelków przekraczających **50 000 punktów**, stosowano losowy subsampling (podpróbkowanie) w celu redukcji rozmiaru danych.

## 2.3. Notatnik 03: Reklasyfikacja semantyczna (`03_data_relabeling.ipynb`)

Oryginalna klasyfikacja danych (7 klas) została uproszczona do 3-klasowego modelu docelowego, optymalnego dla zadania rozpoznawania struktury drzewa.

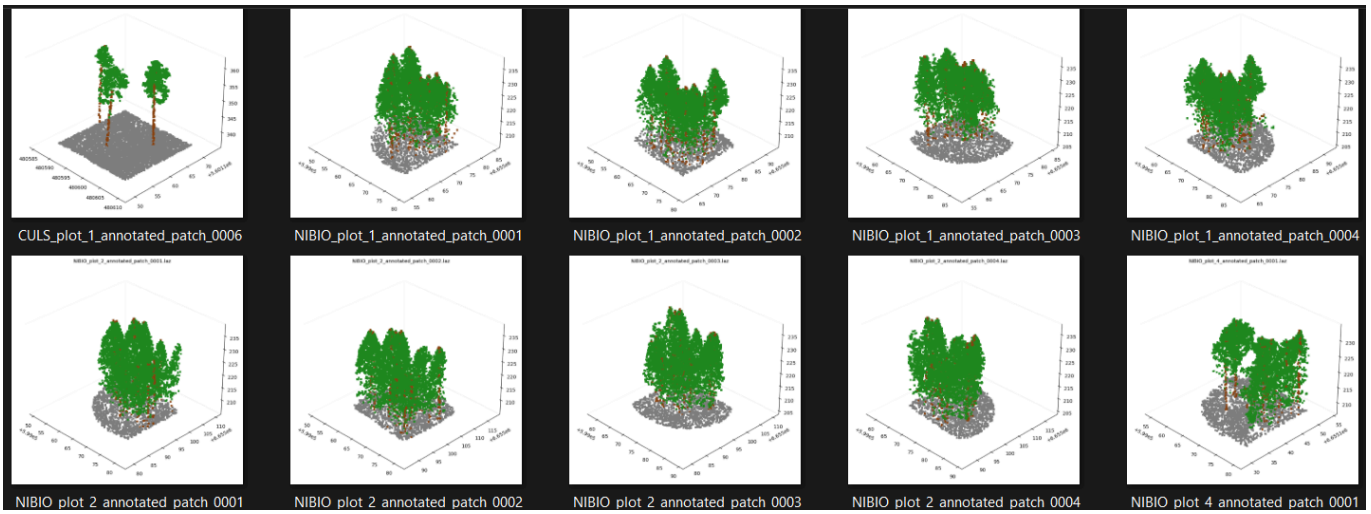
**Schemat mapowania etykiet:**

| Oryginalny kod klasy | Nazwa klasy pierwotnej | Nowy kod klasy | Nazwa klasy docelowej |
|---|---|---|---|
| 0, 1, 2 | Unclassified, Low-veg, Terrain | **0** | **Tło (Background)** |
| 4 | Stem | **1** | **Pień (Trunk)** |
| 5, 6 | Live branches, Woody branches | **2** | **Korona (Tree Crown)** |

*Klasa 3 (Out-points) została trwale usunięta na etapie kafelkowania.*

## 2.4. Notatnik 04: Weryfikacja zbioru danych (`04_dataset_check.ipynb`)

Ostatnim etapem była kontrola jakościowa przetworzonych danych, poprzez generowanie rzutów 3D z użyciem biblioteki Matplotlib. krytpt służy jedynie sprawdzeniu poprawności labelingu/mapowania klas




# 3. Środowisko i proces treningowy

Proces wytrenowania modeli segmentacyjnych został zaprojektowany z myślą o elastyczności i możliwości testowania wielu architektur w ograniczonym budżetowo środowisku obliczeniowym.

## 3.1. Platforma obliczeniowa i infrastruktura

Głównym środowiskiem obliczeniowym była platforma **Paperspace Gradient**. Wybór ten podyktowany był możliwością pracy na "czystym" systemie Ubuntu, co było kluczowe dla poprawnej kompilacji i instalacji specyficznych bibliotek 3D.

- **Główna platforma**: Paperspace (maszyna "Start from Scratch").
- **Wsparcie dodatkowe**: Google Colab (umożliwiono opcję uruchomienia po dostosowaniu ścieżek w notatniku, dotyczy tylko bakcbone PointNext oraz PointNet++).

- **Specyfikacja sprzętowa**:
  - **GPU**: NVIDIA Quadro RTX 5000 (Architektura Turing, sm_75, 16GB VRAM).
  - **CUDA**: Wersja 12.4.
- **Konfiguracja**: Szczegółowy opis przygotowania środowiska (instalacja sterowników i zależności) został zawarty w osobnym dokumencie: `paperspace_environment_setup.md`.

## 3.2. Wykorzystane technologie i biblioteki

Rdzeń systemu treningowego oparto na trzech kluczowych bibliotekach dedykowanych przetwarzaniu chmur punktów i rzadkich struktur danych:

1. **PyTorch Geometric (https://github.com/pyg-team/pytorch_geometric)**: Wykorzystana do większości zadań związanych z przygotowaniem danych, implementacją transformacji przestrzennych, budową Dataloaderów oraz zarządzaniem strukturą grafową danych.
2. **SpConv (https://github.com/traveller59/spconv)**: Zastosowana jako silnik dla backbone oparego na rzadkich splotach 3D.
3. **PointTransformerV3 (https://github.com/pointcept/pointtransformerv3)**: Zastosowana jako silnik dla backbone oparego na PTv3.

## 3.3. Założenia notatnika treningowego

Głównym założeniem opracowanego notatnika było stworzenie modularnego środowiska do szybkiego prototypowania i testowania różnych wariantów sieci neuronowych. System pozwala na łatwą zmianę:

- Architektury sieci (backbone).
- Rozmiaru modelu (liczba kanałów, głębokość sieci).
- Hiperparametrów treningowych w ramach segmentacji semantycznej drzew.

## 3.4. Architektura i konfiguracja eksperymentu (`CFG_DICT`)

System treningowy został w pełni sparametryzowany za pomocą słownika konfiguracyjnego `CFG_DICT`, celem zapewnia powtarzalności eksperymentów orazwyodnego przelaczania się między architekturami backbone.

### 3.4.1. Modułowość Backbone

Notatnik wspiera cztery zróżnicowane pod względem technologicznym architektury:

- **PointNet++**: Wykorzystujący moduły *Set Abstraction* i *Feature Propagation* z ręczną implementacją FPS (*Farthest Point Sampling*) i KNN.
- **PointNeXt**: Wersja PointNet++, wprowadzająca bloki *Inverted Residual* oraz ulepszone osadzenia geometryczne (LGE).
- **SpConv**: Architektura oparta na splotach rzadkich (Sparse Convolutions), zoptymalizowana pod kątem wydajności pamięciowej.
- **PTv3 (Point Transformer v3)**: Model wykorzystujący mechanizm Attention

### 3.4.2. Strategia optymalizacji i funkcje celu

W procesie uczenia zaimplementowano zaawansowane mechanizmy stabilizacji gradientu:

- **Funkcja straty (DiceFocalLoss)**: Hybrydowe połączenie *Focal Loss* (skupiającej się na trudnych punktach, np. pniach) oraz *Dice Loss* (optymalizującej globalne nakładanie się klas/IoU). Zastosowano również *Label Smoothing* (0.15) dla lepszej generalizacji.
- **Optymalizator**: AdamW z wagowym zanikaniem gradientu (*weight decay*).
- **Harmonogram uczenia (Scheduler)**: Wykorzystano `CosineAnnealingLR` poprzedzony 5-epokowym etapem rozgrzewki (*Warmup*).

## 3.5. Augmentacja danych (Domain-Specific Augmentation)

Prawdopodobnie kluczowym elementem projektu jest potok augmentacji, dostosowany do specyfiki danych LiDAR w leśnictwie. Poza standardowymi rotacjami i skalowaniem, zaimplementowano transformacje odzwierciedlające realne warunki terenowe:

- **Random Slope**: Symulacja nachylenia terenu.
- **Semantic Stem Stretch**: Dynamiczne rozciąganie pni drzew przy zachowaniu spójności z koroną.
- **Class Local Density Change**: Losowa redukcja gęstości punktowej w obrębie konkretnych klas (np. rzadsze punkty na pniu), symulująca zmienne warunki skanowania.
- **Forest Geometry Thickener**: Selektywne zagęszczanie punktów klasy "pień" poprzez jittering, mające na celu wzmocnienie reprezentacji geometrycznej najtrudniejszej klasy.
- **Elastic Distortion**: Nieliniowe odkształcenia przestrzenne chmury punktów.

## 3.6. Monitoring i Logging (Weights & Biases)

Proces uczenia jest w pełni zintegrowany z platformą **Weights & Biases (WandB)**. System loguje w czasie rzeczywistym:

- Metryki segmentacji: mIoU, F1-score, Precision i Recall (zarówno globalnie, jak i per-class).
- Wizualizacje 3D:
- Checkpionting: Automatyczne zapisywanie wag modelu dla najlepszego wyniku metryki `val/mIoU`.

## 3.7. Zakres eksperymentów i ograniczenia

Ze względu na pilotażowy charakter projektu oraz ograniczenia czasowe, przeprowadzono selektywną liczbę sesji treningowych. Proces optymalizacji hiperparametrów skupił się na kluczowych parametrach (learning rate, batch size) i dosłownie kilku treningach.

- **PointNet++ i PointNeXt**: Modele te zostały poddane pojedynczym próbom testowym (głównie ze względu na relatywnie długi cza treningu, a w przypadku PointNet++ słąbego wyniku).
- **SpConv i PTv3**: Tutaj dodatkowo zostały wykonane pojedyncze testy wykorzsytujące daaset syntetyczny w trenigu oraz połącenie dataetu syntetycznego z rzeczywistym